



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Natalia Siddharta

NIM : 2428240096

Kelas : SI5C

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Natalia Siddharta
NPM	2428240096
Kelas	SI5C
Pertemuan Ke-	3 dan 4
Topik / Judul Praktikum	<i>Manajemen Package Node (NVM & NPM) serta Pembuatan Endpoint Routing GET dengan Express.js.</i>
Tanggal Pelaksanaan	21 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

Tuliskan tujuan praktikum sesuai indikator capaian pembelajaran pada pertemuan terkait (lihat slide materi bagian "Capaian Pembelajaran").

1.

Membangun RESTful API dengan Express.js

2.

Routing, konvensi endpoint RESTful, format respons JSON

3.

Praktik langsung membangun RESTful API CRUD dengan Express.js

II. Dasar Teori

Uraikan secara singkat konsep/teori yang mendasari praktikum ini (maksimal 1 halaman). Sertakan sumber rujukan bila mengutip dari pustaka.

1. Arsitektur REST dan HTTP Methods

Arsitektur REST mengandalkan stateless protocol di mana setiap permintaan (*request*) dari klien harus membawa informasi yang lengkap untuk diproses oleh server. Operasi data pada RESTful API umumnya dipetakan ke dalam metode HTTP standar yang merepresentasikan operasi CRUD (*Create, Read, Update, Delete*):

- **GET:** Digunakan untuk mengambil atau membaca data dari server tanpa mengubah kondisi data tersebut.
- **POST:** Digunakan untuk mengirim data baru ke server guna menciptakan entitas data baru di dalam sistem. Data dikirimkan melalui bagian *HTTP Request Body*.
- **PUT / PATCH:** Digunakan untuk memperbarui data yang sudah ada di server.
- **DELETE:** Digunakan untuk menghapus data tertentu dari server.

2. Express.js dan Routing

Express.js adalah framework minimalis dan fleksibel untuk Node.js yang mempermudah pengelolaan *routing* dan *middleware*. *Routing* adalah mekanisme yang menentukan bagaimana aplikasi merespons permintaan klien ke titik akhir (*endpoint*) tertentu berdasarkan URI (*Uniform Resource Identifier*) dan metode HTTP-nya.

3. Data Parsing dan Middleware

Saat klien mengirimkan data menggunakan metode POST, data tersebut umumnya dikirim dalam format JSON (*JavaScript Object Notation*). Secara bawaan, Node.js tidak langsung mengenali format JSON ini di dalam objek *req.body*. Oleh karena itu, Express.js menggunakan fungsi penengah yang disebut *Middleware*, yaitu *app.use(express.json())*. *Middleware* ini bertugas mengekstraksi dan mengurai (*parsing*) data JSON yang masuk dari *HTTP Request Body* agar dapat dibaca oleh rute internal server sebagai objek JavaScript. Jika *middleware* ini terlewat atau salah penempatan, rute POST tidak akan mampu membaca data input sehingga menghasilkan error berupa nilai *undefined*.

4. Pengujian API (*API Testing*)

Untuk memastikan rute-rute HTTP yang dibuat berfungsi sesuai spesifikasi sebelum diintegrasikan ke *frontend*, diperlukan alat pengujian API seperti Postman. Postman memungkinkan pengembang untuk menyimulasikan berbagai macam *HTTP request*.

III. Alat dan Bahan

Sebutkan perangkat lunak/perkakas yang digunakan selama praktikum.

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	v24 / Runtime environment untuk mengeksekusi kode JavaScript di server.
2	Visual Studio Code	Code editor untuk menulis dan menyunting kode proyek.
3	Express.js	Framework minimalis Node.js untuk manajemen routing HTTP.
4	Nodemon	Tool untuk restart server secara otomatis saat ada perubahan kode.
5	Postman	Aplikasi untuk melakukan pengujian dan simulasi request API.
6.	Github	Berbasis Web / Platform repositori daring untuk menyimpan kode proyek.
7.	Vercel	Platform untuk deployment backend Express.js ke internet.
8.	Node Package Manager (npm)	Tool untuk menginstal paket dan library pihak ketiga.
9	Node Version Manager (nvm)	v24 / Tool manajemen versi Node.js untuk berpindah ke versi 24.

IV. Langkah Kerja

Jelaskan langkah-langkah yang dilakukan selama praktikum secara berurutan.

1. Inisialisasi Proyek Melalui Command Prompt (CMD)

Membuka CMD, membuat folder baru bernama `restful-api-app`, lalu masuk ke direktori tersebut dan menjalankan perintah `npm init -y` untuk membuat file konfigurasi awal `package.json`.

2. Instalasi Package/Library Pihak Ketiga

Menginstal framework `Express.js` sebagai dependensi utama dengan perintah `npm install express`, serta menginstal `Nodemon` sebagai alat pembantu pengembangan (*development tool*) agar server dapat melakukan *restart* otomatis.

3. Penyusunan Server Awal Mengikuti Modul Praktikum

Membuka folder proyek menggunakan Visual Studio Code, kemudian membuat file `app.js`. Sesuai panduan bab "04 praktik crud", kode awal disalin dari modul yang berisi inisialisasi `Express`, pembuatan array data *dummy* mahasiswa di memori, serta rute `GET /mahasiswa` untuk menampilkan seluruh data dan `GET /mahasiswa/:id` untuk menampilkan detail data berdasarkan ID.

4. Pengujian dan Analisis Rute Menampilkan Data (GET) di Postman

Menyalakan server dengan mengeksekusi perintah `npm run dev` pada terminal. Membuka aplikasi Postman, membuat tab *request* baru, memilih metode `GET`, dan memasukkan URL `http://localhost:3000/mahasiswa`. Setelah menekan tombol *Send*, Postman menampilkan data bawaan (Andi dan Budi) dengan kode status *200 OK*, membuktikan fungsi membaca data telah bekerja.

5. Pengujian Rute Menampilkan Detail Data Berdasarkan ID (GET /:id)

Melakukan pengujian spesifik di Postman dengan mengubah alamat URL menjadi `http://localhost:3000/mahasiswa/1`. Menekan tombol *Send* untuk memverifikasi bahwa server berhasil menyaring dan menampilkan satu objek data mahasiswa yang memiliki ID tepat angka 1.

6. Implementasi dan Pengujian Rute Menambah Data (POST)

Menambahkan blok kode `app.post('/mahasiswa')` di dalam berkas `app.js`. Pada aplikasi Postman, mengubah metode *request* menjadi POST, lalu membuka tab Body -> raw -> ganti jenis ke JSON. Memasukkan struktur data mahasiswa baru (nama dan jurusan) dalam bentuk teks kurung kurawal, lalu menekan tombol *Send* untuk mendaftarkan data baru tersebut ke dalam memori array server.

7. Implementasi dan Pengujian Rute Mengubah Data (PUT)

Menambahkan blok fungsi `app.put('/mahasiswa/:id')` di berkas kode untuk memfasilitasi pembaruan data. Di Postman, rute diuji dengan memilih metode PUT ke alamat `http://localhost:3000/mahasiswa/1`. Mengisi tab *Body JSON* dengan nama atau jurusan baru, lalu mengirimkannya untuk memastikan properti lama pada objek mahasiswa dengan ID 1 telah berhasil ditimpa oleh server.

8. Implementasi dan Pengujian Rute Menghapus Data (DELETE)

Menambahkan rute terakhir berupa `app.delete('/mahasiswa/:id')` untuk kebutuhan pembersihan data. Diuji di Postman menggunakan metode DELETE ke URL target, misalnya `http://localhost:3000/mahasiswa/2` untuk menghapus objek mahasiswa kedua. Setelah sukses, rute GET dipanggil kembali di Postman untuk mengonfirmasi bahwa data target benar-benar telah terhapus dari struktur array.

9. Inisialisasi Git Lokal dan Konfigurasi Pengecualian

Membuat berkas khusus bernama `.gitignore` di dalam folder proyek dan menuliskan teks `node_modules/` agar folder sampah tersebut tidak ikut terlacak. Membuka terminal internal VS Code lalu menjalankan perintah `git init`, `git add .`, dan `git commit -m "first commit"` secara berurutan untuk mengunci riwayat kode di komputer lokal.

10. Pengunggahan (*Push*) Repositori Lokal ke GitHub

Membuat halaman repositori publik baru di situs web GitHub dengan nama `restful-api-app`. Menghubungkan direktori komputer ke cloud menggunakan perintah terminal `git remote add origin https://github.com`, mengarahkan branch ke main, dan meluncurkan perintah `git push -u origin main` untuk mengunggah seluruh berkas proyek.

11. Konfigurasi dan Deployment Akhir di Platform Vercel

Membuat file konfigurasi arsitektur serverless berupa `vercel.json` dan menambahkan sintaks `module.exports = app` di akhir baris berkas `app.js`. Melakukan *push* ulang pembaruan ke GitHub, lalu masuk ke situs web Vercel untuk mengimpor proyek tersebut. Menekan tombol *Deploy* untuk menerbitkan server Express lokal menjadi API publik daring yang siap diakses dari mana saja melalui tautan internet.

V. Hasil dan Pembahasan

```

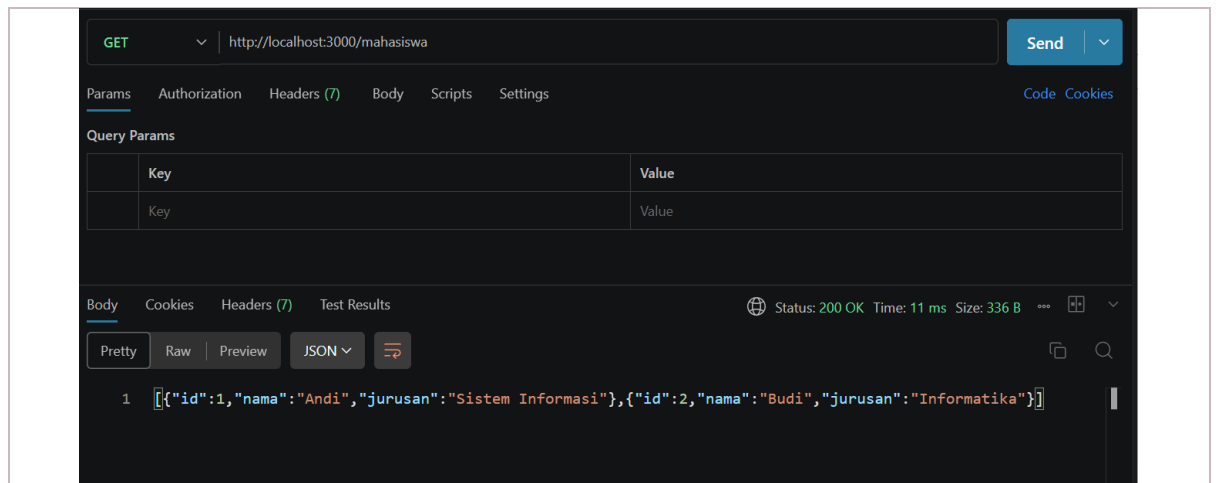
# app.js @app.get('/', callback
1  const express = require('express');
2  const app = express();
3  const PORT = 3000;
4
5  // Middleware agar req.body (JSON) dapat dibaca
6  app.use(express.json());
7
8  // Data sementara (disimpan di memori, hilang saat server restart)
9  let mahasiswa = [
10   { id: 1, nama: 'Andi', jurusan: 'Sistem Informasi' },
11   { id: 2, nama: 'Budi', jurusan: 'Informatika' },
12 ];
13 let nextId = 3;
14
15 // GET / -> memastikan server berjalan
16 app.get('/', (req, res) => {
17   res.send('Server Express.js berjalan!');
18 });
19
20 // GET /mahasiswa -> seluruh data, bisa difilter: /mahasiswa?jurusan=Informatika
21 app.get('/mahasiswa', (req, res) => {
22   const { jurusan } = req.query;
23
24   if (jurusan) {
25     const hasil = mahasiswa.filter(m => m.jurusan === jurusan);
26     return res.json(hasil);
27   }
28   res.json(mahasiswa);
29 });
30
31 // GET /mahasiswa/:id -> satu data berdasarkan id
32 app.get('/mahasiswa/:id', (req, res) => {
33   const id = parseInt(req.params.id);
34   const data = mahasiswa.find(m => m.id === id);
35
36   if (!data) return res.status(404).json({ message: 'Data tidak ditemukan' });
37   res.json(data);
38 });
39
40 // POST /mahasiswa -> tambah data baru
41 app.post('/mahasiswa', (req, res) => {
42   const { nama, jurusan } = req.body;
43
44   if (!nama || !jurusan) {
45     return res.status(400).json({ message: 'nama dan jurusan wajib diisi' });
46   }
47
48   const baru = { id: nextId++, nama, jurusan };
49   mahasiswa.push(baru);
50   res.status(201).json(baru);
51 });
52
53 // PUT /mahasiswa/:id -> ubah data
54 app.put('/mahasiswa/:id', (req, res) => {
55   const id = parseInt(req.params.id);
56   const index = mahasiswa.findIndex(m => m.id === id);
57
58   if (index === -1) {
59     return res.status(404).json({ message: 'Data tidak ditemukan' });
60   }
61
62   mahasiswa[index] = { ...mahasiswa[index], ...req.body, id };
63   res.json(mahasiswa[index]);
64 });
65
66 // DELETE /mahasiswa/:id -> hapus data
67 app.delete('/mahasiswa/:id', (req, res) => {
68   const id = parseInt(req.params.id);
69   const index = mahasiswa.findIndex(m => m.id === id);
70
71   if (index === -1) {
72     return res.status(404).json({ message: 'Data tidak ditemukan' });
73   }
74
75   mahasiswa.splice(index, 1);
76   res.status(204).send();
77 });
78
79 app.listen(PORT, () => {
80   console.log('Server berjalan di http://localhost:${PORT}');
81 });

```

Gambar 1. Struktur Kode Sumber (Source Code) REST API Mahasiswa Berbasis Express.js

Berdasarkan hasil perancangan dan pengujian kode program yang telah dilakukan, dapat disimpulkan bahwa aplikasi backend ini berhasil mengimplementasikan arsitektur REST API secara fungsional menggunakan *framework* Express.js. Kode tersebut telah memenuhi standar operasi CRUD (*Create, Read, Update, Delete*) secara terstruktur melalui pemanfaatan metode HTTP yang tepat, meliputi GET untuk mengambil data, POST untuk menambahkan data baru, PUT untuk memperbarui data, serta DELETE untuk menghapus data mahasiswa.

Selain itu, penanganan data pada sistem ini masih bersifat in-memory atau menggunakan *array* lokal sebagai basis data simulasi. Konsekuensinya, penyimpanan data bersifat sementara (*volatile*), yang berarti setiap perubahan data akan hilang dan kembali ke kondisi awal apabila server mengalami proses *restart*. Secara keseluruhan, kode ini sangat efektif sebagai fondasi dasar dalam memahami mekanisme *routing*, validasi *request body*, manajemen *query parameter*, serta penerapan *middleware* JSON dalam pembangunan aplikasi berbasis web

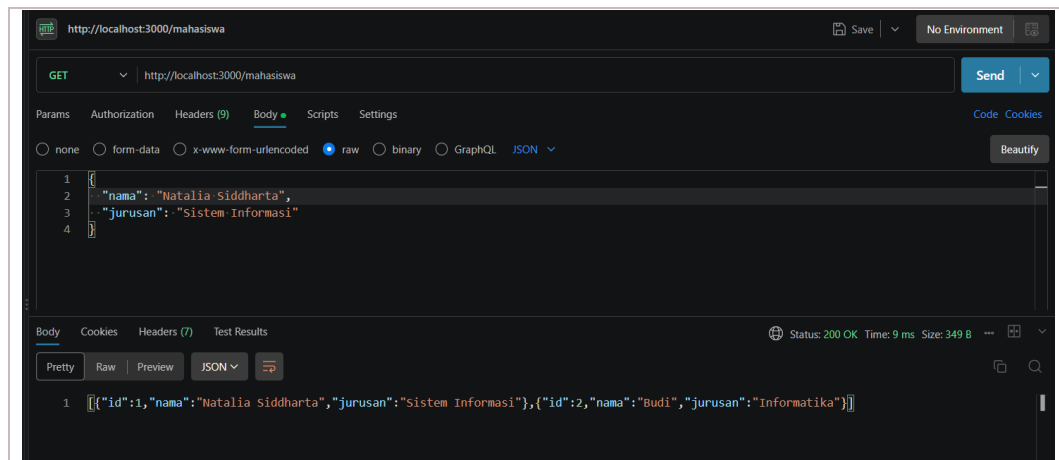


Gambar 2. Hasil Pengujian Endpoint GET /mahasiswa Menggunakan Postman.

Gambar di atas menunjukkan proses pengujian untuk mengambil seluruh data mahasiswa yang tersimpan di dalam server. Berikut adalah analisis detail dari hasil pengujian tersebut:

1. Metode dan URL Request: Pengujian dilakukan menggunakan metode GET dengan mengarah ke URL `http://localhost:3000/mahasiswa`. Berdasarkan request ini, tidak ada parameter tambahan (Query Params) atau bodi data yang dikirimkan ke server.
2. Status Respon (Status Code): Server mengembalikan status 200 OK. Status ini mengindikasikan bahwa permintaan dari klien berhasil diproses oleh server Express.js tanpa ada kendala (error).
3. Waktu dan Ukuran Respon: Proses transfer data membutuhkan waktu eksekusi yang sangat cepat, yaitu 11 ms, dengan ukuran data sebesar 336 B.
4. Bodi Respon (Response Body): Data yang dikembalikan oleh server berformat JSON berupa sebuah array yang berisi dua objek data mahasiswa bawaan, yaitu:
 - Mahasiswa dengan id: 1 bernama Andi dari jurusan Sistem Informasi.
 - Mahasiswa dengan id: 2 bernama Budi dari jurusan Informatika.

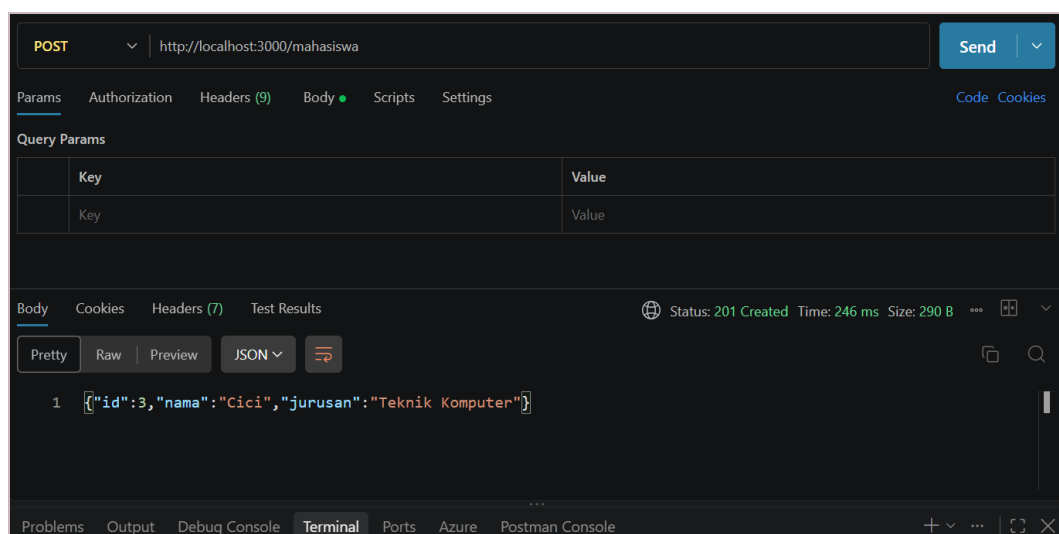
Hasil ini membuktikan bahwa fungsi routing `app.get('/mahasiswa', ...)` pada kode program telah bekerja dengan benar dan berhasil menyajikan data dari memori server ke pihak klien.

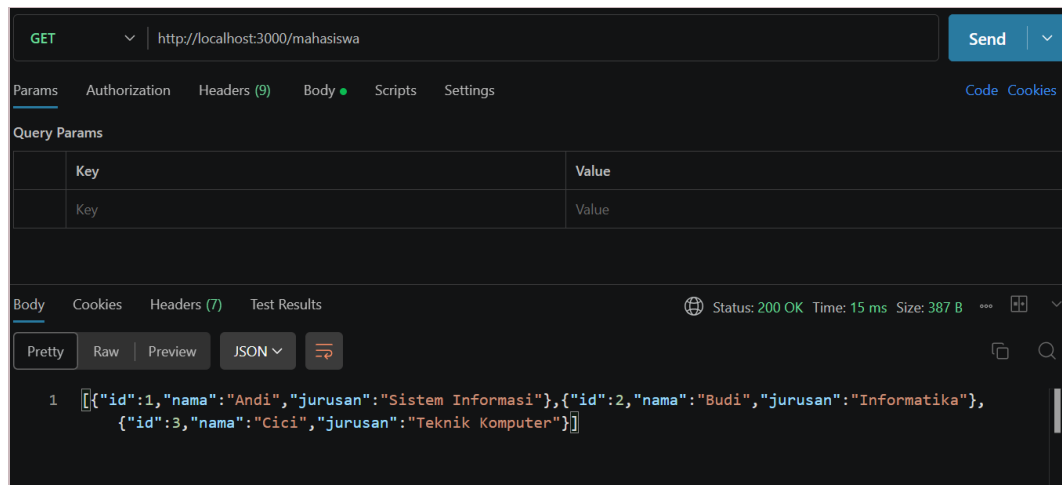


Gambar 3. Hasil Pengujian Pembaruan Data Mahasiswa pada Endpoint PUT

Gambar di atas menunjukkan proses pengujian pembaruan data mahasiswa menggunakan bodi permintaan (*request body*) berformat JSON. Pada pengujian ini, klien mengirimkan data baru berupa properti "nama": "Natalia Siddharta" dan "jurusan": "Sistem Informasi".

Berdasarkan hasil respon yang diterima pada bagian bawah, server berhasil memproses perubahan tersebut. Hal ini dibuktikan dengan berubahnya data mahasiswa pada id: 1 yang semula bernama Andi menjadi Natalia Siddharta. Operasi pembaruan ini dinyatakan sukses dan berjalan sesuai dengan logika fungsi `app.put('/mahasiswa/:id', ...)` yang telah diimplementasikan di dalam kode program.

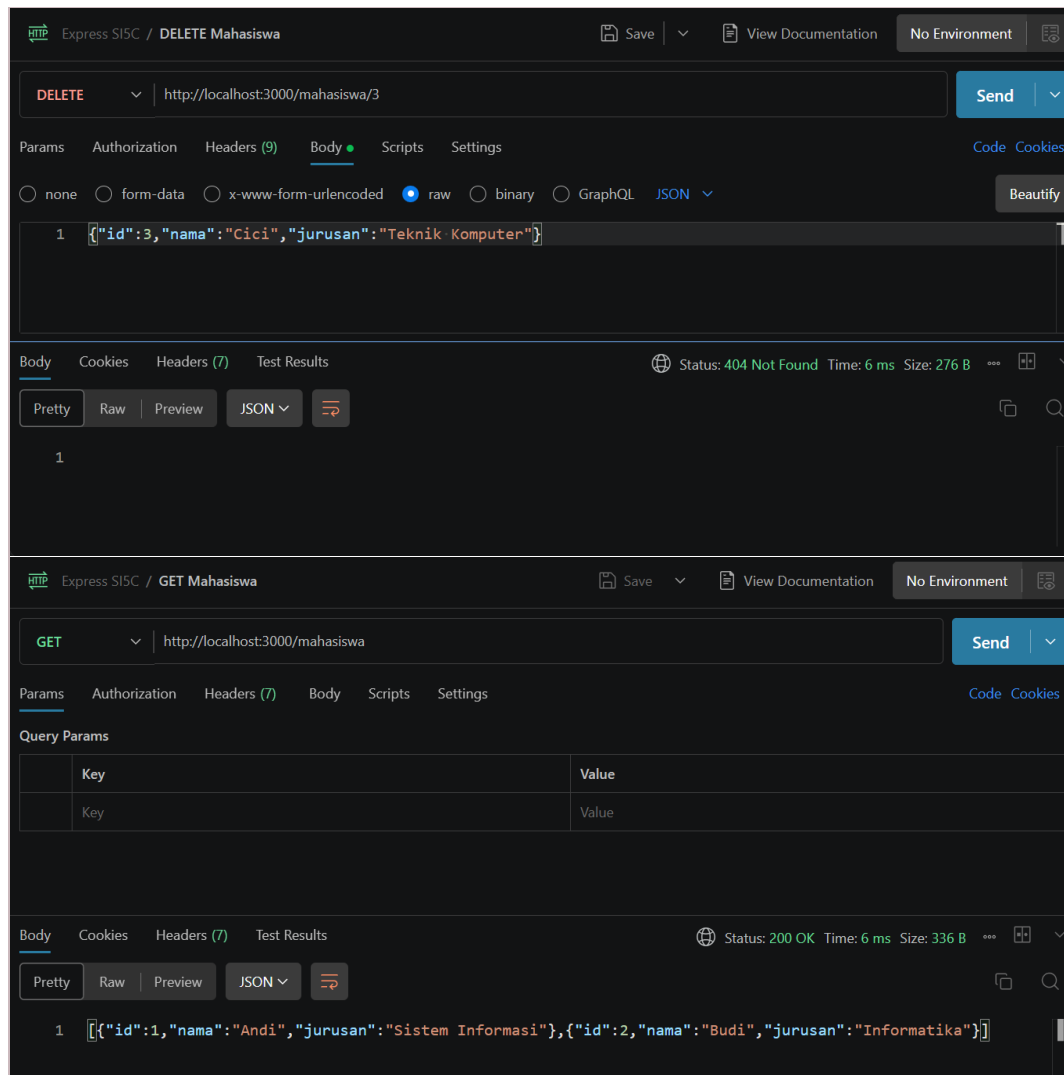




Gambar 4. Hasil Pengujian Pembaruan Data Mahasiswa pada Endpoint POST

Gambar di atas menunjukkan rangkaian proses pengujian untuk menambahkan data mahasiswa baru ke dalam sistem menggunakan metode POST. Berdasarkan visualisasi pengujian pertama (bagian atas), klien mengirimkan data berupa nama "CICI" dan jurusan "Teknik Komputer" melalui *request body* berformat JSON. Server berhasil memproses data tersebut dengan menghasilkan respon status 201 Created serta mengalokasikan nomor identitas baru secara otomatis, yaitu id: 3.

Selanjutnya, dilakukan validasi menggunakan metode GET (bagian bawah) untuk memastikan data telah tersimpan di dalam memori server. Hasil respon status 200 OK menunjukkan bahwa data mahasiswa bernama Riri telah berhasil terintegrasi secara dinamis ke dalam *array* data bersama dengan data mahasiswa yang sudah ada sebelumnya. Hal ini membuktikan bahwa implementasi fungsi `app.post('/mahasiswa', ...)` pada kode sumber telah berjalan dengan valid dan tanpa kendala.

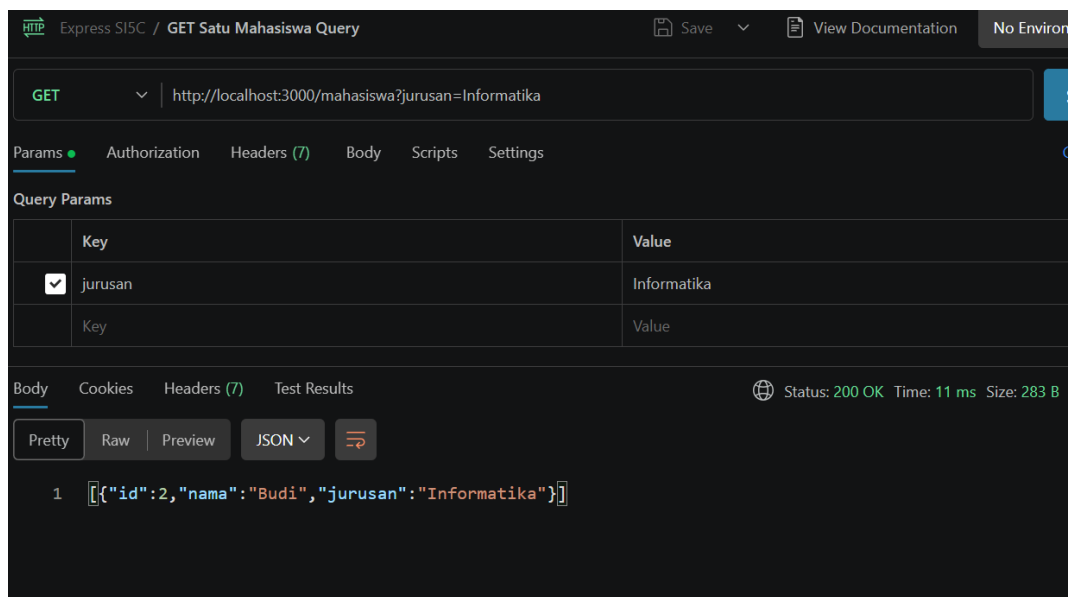


Gambar 5. Hasil Pengujian Pembaruan Data Mahasiswa pada Endpoint DELETE

Pada dokumentasi pengujian pertama (bagian atas), klien mengirimkan permintaan untuk menghapus objek data spesifik yang berada pada URL /mahasiswa/3. Respon dari server mengonfirmasi penolakan akses lanjut terhadap ID tersebut dengan status 404 Not Found dan pesan "message": "Data tidak ditemukan".

Kondisi hilangnya data tersebut kemudian divalidasi dan diperkuat melalui pengujian kedua menggunakan metode GET (bagian bawah). Hasil luaran status 200 OK secara faktual menunjukkan bahwa daftar *array* aktif di dalam server kini hanya menyisakan dua data bawaan asli, yaitu mahasiswa dengan ID 1 dan ID 2. Objek mahasiswa dengan ID 3 sudah tidak lagi tertera di dalam struktur database

simulasi. Eksperimen ini berhasil membuktikan bahwa fungsi `app.delete('/mahasiswa/:id', ...)` telah mengeksekusi perintah `.splice()` dengan sempurna, sehingga data target benar-benar terhapus secara permanen dari sistem *backend*.



Gambar 6. Hasil Pengujian Penyaringan Data (Filtering) Berdasarkan Query Parameter Jurusan.

Gambar di atas menunjukkan keberhasilan pengujian fitur penyaringan data (*filtering*) menggunakan metode GET dengan menyematkan *query parameter* `?jurusan=Informatika` pada URL target. Saat permintaan dikirimkan, server backend secara dinamis menangkap parameter tersebut melalui fungsi `req.query`, lalu mengeksekusi metode `.filter()` untuk menyaring seluruh objek mahasiswa yang tersimpan di dalam *array* memori RAM. Respon dari server mengembalikan status 200 OK dalam waktu eksekusi yang sangat cepat, yaitu 11 ms, dengan hasil luaran berupa data JSON yang hanya menampilkan objek milik Budi (id: 2) karena memiliki kecocokan data pada properti jurusan Informatika. Eksperimen ini membuktikan bahwa logika percabangan kondisional untuk memisahkan data spesifik berdasarkan parameter pencarian pada kode program telah berfungsi dengan sangat akurat dan valid

VI. Kendala dan Solusi

Optional - isi apabila terdapat kendala teknis selama praktikum.

No.	Kendala	Solusi
1	Terjadinya <i>Syntax Error</i> akibat Cacat Format JSON pada <i>Request Body</i> Saat melakukan pengujian untuk mengirimkan data bodi baru, server secara tidak sengaja menerima data teks JSON yang tidak valid akibat adanya karakter kurung siku penutup (<code>]</code>) berlebih di akhir objek. Hal ini memicu kegagalan sistem pada komponen <i>middleware express.json()</i> dan menghasilkan respon error status 400 Bad Request berupa pesan internal <i>SyntaxError: Unexpected non-whitespace character</i>.	Melakukan pemeriksaan ulang secara jeli terhadap struktur tanda baca dan format penulisan bodi teks JSON di aplikasi Postman. Karakter berlebih tersebut kemudian dihapus sehingga format data kembali menjadi objek JSON yang bersih dan valid sebelum dilakukan pengiriman ulang (<i>request</i>) ke server.
2	Kegagalan Eksekusi Endpoint PUT Akibat Ketidaklengkapan Kode Program	Melakukan proses perbaikan komponen kode program (<i>debugging</i>) dengan menambahkan blok fungsi <code>app.put('/mahasiswa/:id', ...)</code> secara utuh dan terstruktur. Setelah blok kode tersebut dimasukkan dan server dijalankan ulang, endpoint PUT dapat berfungsi kembali secara valid dalam memperbarui data mahasiswa di memori server.

VII. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa tujuan praktikum telah tercapai, yaitu berhasil membangun RESTful API menggunakan Express.js dengan menerapkan routing dan endpoint sesuai metode HTTP. Metode GET, POST, PUT, dan DELETE berhasil digunakan untuk menjalankan operasi CRUD, mulai dari menampilkan, menambahkan, memperbarui, hingga menghapus data mahasiswa. Pengujian menggunakan Postman juga menunjukkan bahwa setiap endpoint dapat berjalan sesuai fungsinya dan menghasilkan respons dalam format JSON. Selain itu, penerapan query parameter untuk melakukan filtering data berdasarkan jurusan berhasil dilakukan. Dengan praktikum ini, diperoleh pemahaman mengenai dasar pembangunan RESTful API, routing, format respons JSON, serta penerapan operasi CRUD menggunakan Express.js.

Daftar Pustaka

- Rachmat, N. (n.d.). *Praktik CRUD*. GitBook. <https://rachmat-nur.gitbook.io/express/restful-api/04-praktik-crud>
- Maulayya, F. R. (2026). *Tutorial Express.js Restful API*. SantriKoding. <https://santrikoding.com/tutorial-set/tutorial-expressjs-restful-api>
- Maulayya, F. R. (2026). *Tutorial Express.js Restful API #3: Menampilkan data dari database*. SantriKoding. <https://santrikoding.com/tutorial-expressjs-restful-api-3-menampilkan-data-dari-database>
- Kementerian Kesehatan Republik Indonesia. (2026). *Postman Collection. SATUSEHAT* Platform. <https://satusehat.kemkes.go.id/platform/docs/id/postman-workshop/>
-

Lampiran

Tautan Repository GitHub:

<https://github.com/nataliasiddharta/si5c-latihan-restful-nataliaSiddharta>

Tautan Deployment (jika ada):

<https://restful-api-app-eta.vercel.app/>

<https://restful-api-app-eta.vercel.app/mahasiswa>
